



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y

DISEÑO DIGITAL

CARRERA DE SOFTWARE

TEMA:

Practica Experimental 3

GRUPO:

CAJAS IBARRA IRVIN MARCELO

PANAMA MURILLO MOISES ANTONIO

TAIPE MORA ZAIDA MELISSA

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web – A – [5to Nivel]

Introducción

Esta unidad revisa cuatro temas centrales para entender cómo funcionan las aplicaciones web por dentro: cómo se maneja la información de cada petición y la sesión de un usuario, cómo la aplicación se conecta y trabaja con la base de datos, qué formas existen de organizar el código de un sistema (patrones de arquitectura), y cómo se diseña una aplicación para que soporte más usuarios sin perder rendimiento. Cada tema se explica con apoyo de fuentes académicas recientes, buscando que la teoría quede clara y aplicable a cualquier proyecto de desarrollo web actual.

Tema 1: Objetos Integrados y Gestión de Estado en Aplicaciones Web

1.1 El objeto Request

Cuando un usuario hace clic en un enlace o envía un formulario, el navegador arma una petición HTTP y la manda al servidor. Esa petición se representa en el backend como un objeto Request, que guarda toda la información que llegó del cliente: el método usado (GET, POST, PUT, DELETE), la URL solicitada, los encabezados (headers), los parámetros enviados en la URL o en el cuerpo del mensaje, y las cookies. Los frameworks modernos (como Spring Boot en Java o Laravel en PHP) envuelven esta información en un objeto fácil de usar, para que el programador no tenga que leer el texto crudo del protocolo HTTP.

Una idea importante detrás del objeto Request es que HTTP es un protocolo sin estado: cada petición debe traer toda la información que el servidor necesita para responder, porque el servidor no recuerda nada de peticiones anteriores por sí solo [1]. Esta restricción, llamada comunicación sin estado, es una de las bases del estilo arquitectónico REST y explica por qué el objeto Request tiene que ser autocontenido: si le falta un dato, el servidor no tiene forma de "divinarlo".

1.2 El objeto Response

Del otro lado está el objeto Response, que es la respuesta que arma el servidor después de procesar la petición. Contiene un código de estado (por ejemplo 200 si todo salió bien, 404 si no se encontró el recurso, o 500 si hubo un error interno), encabezados de respuesta, y un cuerpo con los datos solicitados, normalmente en formato JSON en las aplicaciones web

actuales.

El diseño de una buena respuesta no es un detalle menor: una respuesta clara, con el código de estado correcto y una estructura consistente, facilita que el cliente (ya sea una aplicación móvil, un frontend en Angular o React, u otro servicio) sepa cómo interpretar el resultado sin ambigüedades [1]. Request y Response siempre van de la mano: uno no existe sin el otro, y juntos forman el ciclo básico de comunicación de cualquier aplicación web.

1.3 El objeto Session

El objeto Session resuelve un problema que Request y Response no pueden resolver por sí solos: recordar quién es un usuario a lo largo de varias peticiones. Como HTTP no guarda memoria entre peticiones, se necesita un mecanismo aparte para saber, por ejemplo, que un usuario ya inició sesión. Tradicionalmente esto se hacía guardando los datos de sesión en el servidor y enviando al cliente solo un identificador (session ID) dentro de una cookie.

El problema de este modelo es que, si se tienen varios servidores para soportar más tráfico, todas las peticiones de un mismo usuario tendrían que llegar siempre al mismo servidor (esto se conoce como sticky session), lo cual limita el balanceo de carga. Una alternativa moderna es usar tokens JWT (JSON Web Token): el servidor firma un token con la información del usuario y el cliente lo envía en cada petición, sin que el servidor tenga que guardar nada. Para poder invalidar un token antes de que expire (por ejemplo, cuando el usuario cierra sesión), se usa una lista negra guardada en Redis, que revisa si el identificador del token (JTI) ya fue invalidado antes de aceptar la petición [2]. Este enfoque combina lo mejor de los dos mundos: no depende de un solo servidor, pero sigue permitiendo cerrar sesión de forma efectiva.

1.4 Los objetos Application y Page

Además de Request, Response y Session, algunos frameworks web (sobre todo los más clásicos, como JSP en Java o ASP en su versión original) definen dos objetos más. El objeto Application representa toda la aplicación en ejecución: existe una sola instancia mientras el servidor esté encendido, y sirve para guardar información que deben compartir todos los usuarios, como un contador de visitas o una configuración global. El objeto Page, en cambio, tiene el alcance más pequeño de todos: solo vive mientras se procesa una única vista o página, y se usa para datos que no necesitan salir de ese contexto.

En las aplicaciones modernas basadas en APIs REST, como las que se construyen con Spring

Boot o Angular, el concepto de Page casi no se usa, porque ya no existen páginas del lado del servidor en el sentido clásico: el backend solo entrega datos en JSON y es el frontend el que arma la interfaz. El objeto Application, en cambio, sigue teniendo su equivalente en configuraciones globales o valores compartidos que se definen una sola vez al iniciar la aplicación.

Tema 2: Acceso a Datos desde Aplicaciones Web

2.1 Tecnologías de acceso a datos en aplicaciones web

Toda aplicación web necesita comunicarse con una base de datos para guardar y consultar información. Existen varias formas de hacerlo: usar un driver o conector que permite mandar consultas SQL directamente (como JDBC en Java o PDO en PHP), usar un ORM que traduce objetos del lenguaje de programación a tablas de la base de datos, o usar un query builder, que es un punto medio entre escribir SQL a mano y usar un ORM completo. La elección entre estas tecnologías depende de qué tan compleja es la aplicación, qué tanto control se necesita sobre las consultas, y qué tan importante es la velocidad de desarrollo. En proyectos que combinan un backend robusto con un frontend independiente, la capa de acceso a datos suele diseñarse junto con estrategias de optimización como el uso de caché y la paginación de resultados, para que el sistema siga respondiendo rápido aunque crezca la cantidad de datos [3].

2.2 Consultas SQL desde aplicaciones web

La forma más directa de acceder a datos es escribir consultas SQL y enviarlas desde el código de la aplicación. Esto da control total sobre lo que se pide a la base de datos: se puede optimizar cada consulta a mano, usar índices de forma precisa, y evitar que se traiga más información de la necesaria. La desventaja es que el código queda más ligado a un motor de base de datos específico, y cualquier cambio en la estructura de las tablas obliga a revisar todas las consultas escritas a mano. Aun así, para operaciones puntuales de inserción, actualización o eliminación de datos, escribir SQL directo suele ser más rápido de ejecutar que pasar por una capa de abstracción adicional [4].

2.3 Procedimientos almacenados y funciones de BD

Otra opción es mover parte de la lógica de acceso a datos hacia la propia base de datos, usando procedimientos almacenados y funciones. Un procedimiento almacenado es un bloque de código SQL que se guarda dentro del motor de base de datos y que se puede llamar por su nombre desde la aplicación, en lugar de mandar el SQL completo cada vez. Esto tiene ventajas de rendimiento, porque el procedimiento ya está compilado y optimizado por el motor, y también de seguridad, porque se puede restringir el acceso directo a las tablas y obligar a que todo pase por procedimientos controlados. Estudios comparativos entre motores como PostgreSQL y MariaDB muestran que el tiempo de procesamiento de los procedimientos almacenados puede variar según el motor usado, lo que conviene tener en cuenta al elegir la base de datos de un proyecto [4].

2.4 ORM y mapeo objeto-relacional

Un ORM (Object-Relational Mapping) es una herramienta que traduce automáticamente entre objetos del lenguaje de programación y filas de una tabla en la base de datos, para que el programador trabaje con clases y objetos en vez de escribir SQL a mano. Herramientas como Hibernate o Spring Data JPA en Java, o Entity Framework en .NET, son ejemplos comunes de ORM.

La ventaja principal es la productividad: el código queda más limpio y más fácil de mantener. Sin embargo, un ORM mal usado puede generar problemas de rendimiento, siendo el más conocido el problema N+1: ocurre cuando, al pedir una lista de elementos junto con sus datos relacionados, el ORM termina generando una consulta extra por cada elemento de la lista, en lugar de una sola consulta optimizada. Comparaciones entre distintos ORM muestran que existen diferencias notables de rendimiento entre ellos, especialmente en operaciones de escritura o actualización de datos, por lo que elegir bien el ORM y configurarlo correctamente (usando carga anticipada de relaciones cuando sea necesario) es clave para que la aplicación no se vuelva lenta a medida que crece [5].

Tema 3: Patrones de Arquitectura para Aplicaciones Web

3.1 Principios de arquitectura de software

La arquitectura de software es la forma en que se organizan las piezas de un sistema y cómo se relacionan entre sí. Antes de elegir un patrón específico, es importante tener claros algunos

principios generales: separar responsabilidades (que cada parte del sistema haga una sola cosa bien definida), reducir el acoplamiento entre componentes (para que un cambio en una parte no obligue a modificar todas las demás), y buscar que el sistema sea fácil de entender y de mantener con el tiempo. Estos principios son la base para clasificar y comparar los distintos estilos arquitectónicos que existen, desde los más simples (como la arquitectura en capas) hasta los más distribuidos (como los microservicios) [6].

3.2 Arquitectura por capas

La arquitectura por capas organiza el sistema en niveles, donde cada capa solo se comunica con la capa que tiene justo encima o justo debajo. Una distribución típica en aplicaciones web es: capa de presentación (interfaz de usuario), capa de negocio (reglas y lógica de la aplicación) y capa de datos (acceso a la base de datos). Este patrón es fácil de entender, fácil de probar por partes, y es el punto de partida natural para la mayoría de aplicaciones nuevas, sobre todo cuando el equipo de desarrollo es pequeño y se busca avanzar rápido sin la complejidad extra de un sistema distribuido [6].

3.3 Arquitecturas en pizarra, dirigida por eventos y peer-to-peer

Existen otros estilos arquitectónicos pensados para problemas distintos a los que resuelve la arquitectura por capas. La arquitectura en pizarra (blackboard) usa un espacio de datos compartido al que varios componentes independientes pueden leer y escribir, sin comunicarse directamente entre ellos; es común en sistemas de inteligencia artificial donde varias fuentes de conocimiento colaboran para resolver un problema. La arquitectura dirigida por eventos (event-driven) organiza el sistema alrededor de eventos que se producen, se detectan y se consumen: un componente genera un evento (por ejemplo, "se creó un pedido") y otros componentes reaccionan sin necesidad de conocerse entre sí, lo que reduce el acoplamiento. Por último, la arquitectura peer-to-peer (P2P) elimina la idea de un servidor central: todos los nodos participan como iguales, compartiendo recursos directamente entre ellos, lo que mejora la tolerancia a fallos pero complica la coordinación general del sistema [6].

3.4 Arquitectura Orientada a Servicios (SOA) y Microservicios

La Arquitectura Orientada a Servicios (SOA) divide un sistema grande en servicios independientes que se comunican mediante protocolos estándar, permitiendo que cada servicio se desarrolle y se despliegue por separado. Los microservicios llevan esta idea un paso más allá:

en lugar de tener pocos servicios grandes, se construyen muchos servicios pequeños, cada uno enfocado en una sola función del negocio, con su propia base de datos si es necesario. Esta forma de organizar el sistema permite escalar cada servicio de manera independiente y facilita que distintos equipos trabajen en paralelo sin bloquearse unos a otros [6].

Sin embargo, esta libertad tiene un costo: coordinar la comunicación entre tantos servicios, mantener la consistencia de los datos, y gestionar el despliegue de todos ellos añade una complejidad operacional que no existe en una arquitectura monolítica. Por eso, la decisión de usar microservicios en lugar de una arquitectura por capas debe basarse en el tamaño real del proyecto y del equipo, y no adoptarse solo porque es una tendencia popular [7].

Tema 4: Diseño de Arquitecturas Web Escalables

4.1 Escalabilidad en aplicaciones web

La escalabilidad es la capacidad de un sistema para seguir funcionando bien cuando aumenta la cantidad de usuarios o de datos que debe manejar. Existen dos formas principales de escalar: la escalabilidad vertical, que consiste en agregar más recursos a un mismo servidor (más memoria, más procesador), y la escalabilidad horizontal, que consiste en agregar más servidores y repartir la carga de trabajo entre todos ellos. La escalabilidad vertical es más sencilla de implementar pero tiene un límite físico, mientras que la escalabilidad horizontal es más compleja de configurar (requiere balanceo de carga y sincronización entre servidores) pero, en teoría, no tiene un techo tan cercano. Comparaciones entre arquitecturas monolíticas y basadas en microservicios muestran que el estilo elegido influye directamente en qué tan fácil o difícil resulta escalar el sistema más adelante, sobre todo cuando el tráfico crece de forma importante [8].

4.2 Patrones de caché en aplicaciones web

La caché es una técnica que guarda temporalmente los resultados de una operación costosa (como una consulta a la base de datos) para no tener que repetirla cada vez que se necesita el mismo dato. Existen varios patrones para usar la caché: en el patrón cache-aside, la aplicación primero busca el dato en la caché y, si no lo encuentra, lo busca en la base de datos y después lo guarda en la caché para la próxima vez; en el patrón write-through, cada vez que se guarda un dato nuevo, se escribe al mismo tiempo en la base de datos y en la caché, manteniéndolas siempre sincronizadas.

Usar caché de forma correcta puede mejorar mucho el tiempo de respuesta de una aplicación, pero también introduce nuevos retos: hay que decidir cuánto tiempo debe durar un dato guardado antes de considerarse viejo (TTL), y cómo evitar que muchas peticiones lleguen a la vez a buscar un dato que justo venció (un problema conocido como cache stampede). Diseñar bien estas reglas de invalidación es tan importante como decidir qué datos cachear [9].

4.3 Documentación y evaluación de arquitecturas web

Diseñar una arquitectura escalable no sirve de mucho si nadie más en el equipo entiende cómo está construida ni por qué se tomaron ciertas decisiones. Por eso, documentar la arquitectura es una parte tan importante del trabajo como escribir el código. Una forma común de hacerlo es usando diagramas que muestren distintos niveles de detalle, desde una vista general del sistema hasta los componentes internos de cada parte, de manera que personas con distintos roles (desarrolladores, gerentes, otros equipos) puedan entender la arquitectura según lo que necesiten saber. Además de documentar cómo quedó construido el sistema, también es importante evaluar si la arquitectura elegida realmente cumple lo que se esperaba de ella, comparando el rendimiento antes y después de aplicar ciertos cambios, para tomar decisiones basadas en datos reales y no solo en suposiciones [3].

5. Fundamento Teórico Aplicado al PFC

Esta sección desarrolla los mismos cuatro ejes teóricos revisados en la Unidad III, pero llevados a la práctica concreta del proyecto: se agregan tablas comparativas, código real y decisiones de diseño tomadas para este sistema en particular.

5.1 Gestión de Estado en Aplicaciones Web: Request, Response, Session

La naturaleza stateless de HTTP y sus implicaciones. HTTP es un protocolo sin estado por diseño: cada petición que llega al servidor se procesa de forma aislada, sin memoria de peticiones anteriores. Esto es una ventaja para la escalabilidad, porque cualquier servidor puede atender cualquier solicitud sin depender de información guardada previamente en otro servidor. Sin embargo, es un desafío para las aplicaciones que sí necesitan recordar algo del usuario entre una petición y otra, como el carrito de compras o el hecho de haber iniciado sesión. Para resolver esto existen distintos mecanismos de estado: las cookies, que el navegador guarda y reenvía automáticamente; las sesiones del lado del servidor, donde el servidor guarda los datos y solo entrega un identificador al cliente; los tokens JWT, que llevan la información del usuario firmada dentro del propio token, sin que el servidor tenga que guardar nada; y el uso directo de una base de datos para persistir estados más complejos. La comunicación sin estado es, de hecho, una de las restricciones formales del estilo arquitectónico REST [1].

Comparativa de mecanismos de almacenamiento client-side.

Tabla 1: Comparativa de mecanismos de almacenamiento client-side

Criterio	localStorage	sessionStorage	Cookies
Capacidad	~5-10 MB	~5-10 MB	~4 KB
Persistencia	Permanente	Solo la pestaña	Configurable (TTL)
Accesible desde JS	Sí	Sí	Solo si no es HttpOnly
Envío automático HTTP	No	No	Sí (con cada petición)
Riesgo XSS	Alto	Alto	Bajo (si HttpOnly)
Riesgo CSRF	No aplica	No aplica	Sí (mitigable con SameSite)

En este proyecto se implementó una cookie HttpOnly para almacenar el JWT de sesión (AuthController), lo que la hace inaccesible desde JavaScript y mitiga ataques XSS. El atributo Secure está desactivado en el entorno de desarrollo (solo se activaría en producción bajo HTTPS). Como trabajo futuro queda pendiente fijar explícitamente el atributo SameSite=Strict para reforzar la mitigación de CSRF, ya que actualmente la cookie depende

del valor por defecto del navegador. `sessionStorage` se reserva para datos temporales de formularios que no necesitan persistir entre pestañas.

Sesiones escalables con Redis. Cuando una aplicación se despliega en varios servidores para soportar más tráfico, el modelo tradicional de sesión en memoria de un solo servidor deja de funcionar bien: si el balanceador de carga manda la siguiente petición de un usuario a otro servidor distinto, ese servidor no tiene la sesión guardada. La solución típica y poco escalable es forzar el llamado sticky session, donde todas las peticiones de un usuario se dirigen siempre al mismo servidor, pero esto limita el balanceo de carga y crea un punto único de fallo. La alternativa es centralizar el almacenamiento de sesión en un sistema externo que todos los servidores puedan consultar por igual, siendo Redis una de las opciones más usadas para esto por su velocidad de acceso en memoria [10]. Comparando las tres alternativas de almacenamiento de sesión: guardar en el sistema de archivos es lento y no escala entre varios servidores; guardar en una base de datos relacional es más confiable pero más lento por el overhead de las consultas SQL; guardar en Redis combina velocidad (al ser en memoria) con la posibilidad de que cualquier servidor lo consulte, aunque exige aceptar que los datos son menos duraderos que en una base de datos tradicional si Redis no se configura con persistencia.

Objetos Request y Response en el framework del PFC.

El siguiente fragmento muestra el ciclo completo de una petición en Spring Boot, desde que llega el objeto Request hasta que se construye el Response:

```
1 @RestController
2 @RequestMapping("/api/entidad")
3 public class EntidadController {
4
5     @Autowired
6     private EntidadService entidadService;
7
8     @GetMapping
9     public ResponseEntity<Page<EntidadDTO>> listar(
10         @RequestParam(defaultValue = "0") int page,
11         @RequestParam(defaultValue = "10") int size,
12         @RequestParam(required = false) String filtro) {
13
14         Pageable pageable = PageRequest.of(page, size);
15         Page<EntidadDTO> resultado = entidadService.listar(filtro,
16             pageable);
```

```

16         return ResponseEntity.ok(resultado);
17     }
18 }

```

Aquí, el objeto `Request` llega implícito a través de las anotaciones `@RequestParam`, que extraen los parámetros de la URL (página, tamaño, filtro). El objeto `Response` se construye con `ResponseEntity.ok(...)`, que arma el código de estado 200 y el cuerpo en formato JSON con los datos paginados.

5.2 ORM y Mapeo Objeto-Relacional

El problema del impedance mismatch. El modelo relacional organiza la información en tablas, filas y relaciones entre tablas (joins), mientras que el modelo orientado a objetos organiza la información en clases, con herencia y asociaciones entre objetos. Estos dos paradigmas se diseñaron con objetivos distintos y no encajan de forma natural entre sí: por ejemplo, una tabla no tiene un concepto nativo de herencia, mientras que en programación orientada a objetos la herencia es central. Esta incompatibilidad se conoce como impedance mismatch, y estudios recientes muestran que sigue generando problemas reales de rendimiento en los frameworks ORM actuales, incluso después de años de mejoras en estas herramientas [11]. Para resolver el problema específico de la herencia, existen tres estrategias de mapeo: single table inheritance, donde todas las subclases se guardan en una sola tabla con una columna que indica el tipo (rápido de consultar, pero con muchas columnas vacías); class table inheritance, donde cada clase (incluyendo las clases base) tiene su propia tabla relacionada por clave foránea (más normalizado, pero requiere más joins); y concrete table inheritance, donde cada subclase concreta tiene su propia tabla completa, sin tabla para la clase base (evita joins, pero duplica columnas). La elección depende de qué tan profunda es la jerarquía y qué tan frecuentes son las consultas polimórficas.

Comparativa ORM vs. SQL puro vs. query builder.

Tabla 2: Comparativa ORM vs. SQL puro vs. query builder

Criterio	ORM (JPA)	SQL puro	Query Builder
Curva de aprendizaje	Media-alta	Baja	Media
Rendimiento (consultas simples)	Bueno	Óptimo	Bueno
Rendimiento (consultas complejas)	Riesgo N+1	Óptimo (manual)	Bueno
Portabilidad entre SGBD	Alta	Baja	Media-alta
Facilidad de migración de esquema	Alta (con Flyway)	Manual	Media
Debugging/logging de queries	Requiere activar logs	Directo	Directo

Herramientas como Hibernate o Spring Data JPA muestran diferencias medibles de rendimiento frente al SQL escrito a mano, sobre todo en operaciones de escritura y actualización, lo que debe considerarse al elegir la tecnología de acceso a datos según la naturaleza del proyecto [5].

El problema N+1 y lazy/eager loading.

Para ilustrar el problema N+1, se toma como ejemplo el listado de Entidad junto con su relación @ManyToOne hacia Categoria, configurada con FetchType.LAZY:

```
1 @ManyToOne(fetch = FetchType.LAZY)
2 @JoinColumn(name = "categoria_id", nullable = false)
3 private Categoria categoria;
```

Al ser LAZY, Hibernate no trae la Categoria junto con la Entidad en la consulta inicial; la trae recién cuando se accede al getter. El siguiente código dispara el problema N+1:

```
1 // Sin eager loading: dispara 1 query para la lista + N queries
   // adicionales
2 List<Entidad> lista = entidadRepository.findAll();
3 lista.forEach(e -> e.getCategoria().getNombre()); // 1 query extra
   // por cada elemento
```

Con 55 registros de prueba cargados por Flyway (V3 datos semilla.sql), este código genera 1 query inicial más 55 queries adicionales (una por cada acceso a getCategoria()), es decir, 56 queries en total. La solución es usar carga anticipada mediante JOIN FETCH, que trae la relación en la misma consulta sin cambiar el FetchType declarado en la entidad:

```
// Con eager loading: dispara 1 sola query con JOIN
@Query("SELECT e FROM Entidad e JOIN FETCH e.categoria")
List<Entidad> findAllWithCategoria();
```

Activando `spring.jpa.show-sql=true` en `application.properties`, se confirma que el número de queries baja de 56 a 1 al usar JOIN FETCH, una reducción medible y significativa en la carga enviada a la base de datos.

Migraciones y gestión del esquema. Existen tres formas principales de gestionar los cambios en el esquema de una base de datos a lo largo del tiempo: las migraciones versionadas (como Flyway o Liquibase), donde cada cambio queda registrado en un archivo con número de versión y se aplica de forma ordenada y repetible; la generación automática del esquema a

partir de las entidades (por ejemplo `hbm2ddl.auto=update` en Hibernate), que es rápida para desarrollo pero riesgosa en producción porque puede borrar o modificar datos sin previo aviso; y las herramientas dedicadas de gestión de esquema, que ofrecen más control sobre rollback y validación previa a aplicar los cambios. Para un entorno de producción, lo recomendable es usar migraciones versionadas, ya que dejan un historial claro y reproducible de cómo evolucionó la base de datos, mientras que la generación automática debe reservarse solo para el entorno de desarrollo.

5.3 Patrones de Arquitectura para Aplicaciones Web Escalables

Modelo C4 para documentar arquitectura. El Modelo C4, propuesto por Simon Brown, organiza la documentación de una arquitectura de software en cuatro niveles de detalle progresivo. El nivel 1 (Contexto) muestra el sistema como una caja única y sus relaciones con los usuarios y sistemas externos, pensado para audiencias no técnicas. El nivel 2 (Contenedores) abre esa caja y muestra las aplicaciones y bases de datos que la componen, junto con la tecnología usada en cada una. El nivel 3 (Componentes) entra a un contenedor específico y muestra los módulos internos que lo forman. El nivel 4 (Código) llega hasta el detalle de clases, y solo se recomienda usarlo en las partes más críticas del sistema. La razón por la que C4 suele preferirse sobre UML de alto nivel para comunicar arquitectura es que permite mostrar exactamente el nivel de detalle que cada audiencia necesita, evitando saturar a un directivo con detalles de implementación o dejar a un desarrollador sin la información técnica que necesita [12].

Arquitectura por capas vs. microservicios — análisis para el PFC. La arquitectura por capas separa el sistema en presentación, negocio y datos, con un despliegue único y sencillo, ideal para equipos pequeños que buscan avanzar rápido. Los microservicios, en cambio, dividen el sistema en servicios pequeños e independientes, lo que permite escalar cada uno por separado, pero suma una complejidad operacional considerable: hay que coordinar el despliegue de varios servicios, manejar la comunicación entre ellos y lidiar con las llamadas falacias de los sistemas distribuidos (por ejemplo, asumir que la red siempre es confiable o que la latencia es cero) [6].

Para este PFC, con un equipo de tres integrantes y un plazo académico limitado, se optó por una arquitectura monolítica por capas (**Controller** → **Service** → **Repository** → **Entity**), tal como se documenta en el ADR-001. Esta decisión se basa en el concepto de team cognitive load: un equipo pequeño no puede sostener de forma sana la complejidad operacional que exige coordinar el despliegue y la comunicación entre varios microservicios

independientes, sumado al riesgo de caer en las llamadas falacias de los sistemas distribuidos. Estudios recientes que aplican metodologías de decisión multicriterio confirman que la arquitectura monolítica obtiene mayor puntaje de utilidad cuando se prioriza el rendimiento y la simplicidad operacional sobre la escalabilidad extrema [13].

Comparaciones recientes entre arquitecturas monolíticas y de microservicios confirman que la decisión debe basarse en el tamaño real del equipo y del proyecto, no en la popularidad del patrón [7], [8].

Event-Driven Architecture (EDA) y casos de uso web. En una arquitectura dirigida por eventos, un componente productor genera un evento (por ejemplo, "se creó una nueva cita") y lo publica en un event bus como RabbitMQ o Apache Kafka; otros componentes, los consumidores, escuchan esos eventos y reaccionan sin necesidad de conocer quién los generó, lo que reduce el acoplamiento entre partes del sistema [6]. Un caso particular de EDA en aplicaciones web son los WebSockets, que permiten comunicación bidireccional en tiempo real: la conexión comienza como una petición HTTP normal y luego se actualiza (protocol upgrade) a una conexión persistente por la que el servidor puede enviar datos al cliente sin que este los pida explícitamente. Usar EDA o WebSockets se justifica cuando el sistema necesita notificar cambios en tiempo real (por ejemplo, avisar a un veterinario que llegó una nueva cita sin que tenga que refrescar la página); si la comunicación puede esperar el ciclo normal de petición-respuesta, una API REST síncrona sigue siendo la opción más simple y suficiente.

ADR como práctica de gobernanza arquitectónica. Documentar una decisión arquitectónica no es solo registrar qué se eligió, sino también por qué se eligió, qué alternativas se consideraron y qué consecuencias se aceptan a cambio. Esta práctica, formalizada por Michael Nygard bajo el nombre de Architecture Decision Record (ADR), se ha vuelto una costumbre extendida en equipos de desarrollo modernos porque evita que el conocimiento detrás de una decisión se pierda con el tiempo o cuando cambia el equipo. Estudios sobre el uso real de ADR en proyectos de código abierto muestran que su adopción ha crecido de forma sostenida en los últimos años, y que los equipos que los usan reportan mayor facilidad para justificar y revisar decisiones pasadas [12], [14].

5.4 Caché: Patrones y Medición de Rendimiento

Niveles de caché en una aplicación web. La caché puede aplicarse en distintos niveles de una aplicación web, cada uno con un propósito distinto. La caché del navegador usa encabezados como Cache-Control, ETag y Last-Modified para evitar que el navegador vuelva

a pedir un recurso que no ha cambiado. La CDN distribuye contenido estático (imágenes, archivos CSS o JS) en servidores cercanos geográficamente al usuario. La caché de aplicación, usando Redis o Memcached, guarda resultados de operaciones costosas para no repetirlas. La caché de base de datos, como el buffer pool de MySQL, mantiene en memoria las páginas de datos más consultadas. Cada nivel necesita definir qué datos cachear, cuánto tiempo deben durar (TTL) antes de considerarse vencidos, y cómo invalidarlos cuando cambian [9].

Patrones de caché. El patrón cache-aside es el más común: la aplicación primero busca el dato en la caché, y si no lo encuentra (cache miss), lo busca en la base de datos y lo guarda en la caché para la próxima consulta. El patrón read-through delega esa lógica a la propia capa de caché, que sabe cómo ir a buscar el dato si no lo tiene. El patrón write-through escribe al mismo tiempo en la base de datos y en la caché, manteniéndolas siempre sincronizadas, a costa de una escritura más lenta. El patrón write-behind escribe primero en la caché y de forma asíncrona actualiza la base de datos después, ganando velocidad de escritura pero arriesgando la consistencia si el sistema falla antes de sincronizar. Un problema conocido de estos patrones es el cache stampede: cuando un dato muy solicitado vence al mismo tiempo, muchas peticiones simultáneas intentan recalcularlo a la vez, sobrecargando la base de datos. Este problema se mitiga con un mutex que permite que solo una petición recalcule el dato mientras las demás esperan, o con probabilistic early expiration, donde el dato se renueva un poco antes de vencer, de forma aleatoria, para evitar que todas las peticiones lleguen exactamente al mismo tiempo [9].

Redis: estructura de datos y comandos fundamentales. Redis no es solo un almacén simple de clave-valor: soporta varias estructuras de datos, entre ellas strings, hashes, lists, sets, sorted sets y streams, lo que permite modelar distintos tipos de información de forma eficiente [10].

El siguiente código muestra las dos responsabilidades de Redis definidas en el ADR-003, implementadas en `EntidadService` (caché) y `TokenBlacklistService` (blacklist de JWT).

```
1 // (a) Cache-aside sobre el listado principal      EntidadService
2 @Cacheable(value = "entidad-listado", key = "#filtro + '-' + #page")
3 public Page<EntidadDTO> listar(String filtro, Pageable pageable) {
4     return entidadRepository.buscar(filtro, pageable);
5 }
6
7 @CacheEvict(value = "entidad-listado", allEntries = true)
8 public void guardar(Entidad entidad) {
9     entidadRepository.save(entidad);
```

```

10 }
11
12 // (b) Blacklist de JTI al hacer logout - TokenBlacklistService
13 public void agregarABlacklist(String jti, long segundosRestantes) {
14     redisTemplate.opsForValue()
15         .set("blacklist:" + jti, "true", segundosRestantes, TimeUnit.
16         SECONDS);
17 }
18
19 public boolean estaEnBlacklist(String jti) {
20     return redisTemplate.hasKey("blacklist:" + jti);
21 }

```

La verificación de la blacklist se ejecuta en `JwtAuthFilter`, antes de aceptar cualquier petición autenticada, consultando `TokenBlacklistService.estaEnBlacklist(jti)` extraído del token mediante `JwtProvider`.

Metodología de benchmarking de rendimiento web. Para medir el impacto real de la caché de forma estadísticamente válida, no basta con correr la consulta una sola vez con y sin caché: se necesitan al menos 10 repeticiones de cada escenario para poder calcular una media y una desviación estándar representativas, verificar que la diferencia observada es significativa y no solo ruido aleatorio, y aislar la variable que se está midiendo (usando siempre la misma consulta, bajo las mismas condiciones de red y de carga del sistema) [3].

Siguiendo la metodología descrita, se ejecutó la consulta principal de listado 10 veces con caché deshabilitada y 10 veces con caché habilitada, midiendo el tiempo con `System.nanoTime()` en cada llamada.

Tabla 3: Resultados del benchmark de caché ($S = T_{sin}/T_{con}$)

Métrica	Sin Redis (ms)	Con Redis (ms)
Promedio	45,49	14,36
P95	63,33	17,76

El speedup calculado fue $S = T_{sin}/T_{con} = 45,49/14,36 = 3,17x$. Este resultado supera el umbral de referencia $S > 2$, por lo que sí justifica la complejidad operacional adicional de incorporar Redis al stack. Además, el patrón cache-aside implementado es de tipo fail-open: si Redis no responde, la petición simplemente cae a consultar PostgreSQL directamente, por lo que una caída de Redis degrada el rendimiento pero no compromete la disponibilidad del CRUD. Esta decisión queda documentada en el ADR-003.

Referencias

- [1] S. Kotstein y J. Bogner, *Which RESTful API Design Rules Are Important and How Do They Improve Software Quality? A Delphi Study with Industry Experts*, 2021. [10.48550/arXiv.2108.00033](https://arxiv.org/abs/10.48550/arXiv.2108.00033). arXiv: [2108.00033](https://arxiv.org/abs/2108.00033). dirección: <https://arxiv.org/abs/2108.00033>.
- [2] S. Dalimunthe, E. Hasri Putra y M. A. Fadhly Ridha, “Restful API Security Using JSON Web Token (JWT) With HMAC-Sha512 Algorithm in Session Management,” *IT Journal Research and Development*, vol. 8, n.º 1, págs. 81-94, 2023. [10.25299/itjrd.2023.12029](https://doi.org/10.25299/itjrd.2023.12029).
- [3] M. Menghnani, “A Comprehensive Survey on Scalability and Performance in Full Stack Web Applications,” *International Journal of Advanced Research in Science, Communication and Technology*, vol. 5, n.º 12, 2025. [10.48175/IJARSCT-25930](https://doi.org/10.48175/IJARSCT-25930).
- [4] N. Hartono y Erfina, “Comparison of Stored Procedures on Relational Database Management System,” *Tech-E*, vol. 4, n.º 2, 2021. [10.31253/te.v4i2.529](https://doi.org/10.31253/te.v4i2.529).
- [5] A. E. Güvercin y B. Avenoglu, “Performance Analysis of Object-Relational Mapping (ORM) Tools in .Net 6 Environment,” *Bilişim Teknolojileri Dergisi*, vol. 15, n.º 4, págs. 453-465, 2022. [10.17671/gazibtd.1059516](https://doi.org/10.17671/gazibtd.1059516).
- [6] M. Karimi et al., *Proposing a Dynamic Executive Microservices Architecture Model for AI Systems*, 2023. [10.48550/arXiv.2308.05833](https://arxiv.org/abs/10.48550/arXiv.2308.05833). arXiv: [2308.05833](https://arxiv.org/abs/2308.05833). dirección: <https://arxiv.org/abs/2308.05833>.
- [7] V. Velepucha y P. Flores, “A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,” *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. [10.1109/ACCESS.2023.3305687](https://doi.org/10.1109/ACCESS.2023.3305687).
- [8] G. Blinowski, A. Ojdowska y A. Przybylek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, 2022. [10.1109/ACCESS.2022.3152803](https://doi.org/10.1109/ACCESS.2022.3152803).
- [9] J. Mertz, I. Nunes, L. Della Toffola, M. Selakovic y M. Pradel, *Satisfying Increasing Performance Requirements with Caching at the Application Level*, 2020. [10.48550/arXiv.2010.12939](https://arxiv.org/abs/10.48550/arXiv.2010.12939). arXiv: [2010.12939](https://arxiv.org/abs/2010.12939). dirección: <https://arxiv.org/abs/2010.12939>.
- [10] L. Shi, H. Qiao, C. Yang, Y. Jiang, K. Yu y C. Chen, “Research and Application of Distributed Cache Based on Redis,” *Journal of Software*, vol. 19, n.º 1, págs. 1-8, 2024. [10.17706/jsw.19.1.1-8](https://doi.org/10.17706/jsw.19.1.1-8).

- [11] D. Colley, C. Stanier y M. Asaduzzaman, “Investigating the Effects of Object-Relational Impedance Mismatch on the Efficiency of Object-Relational Mapping Frameworks,” *Journal of Database Management*, vol. 31, n.º 4, págs. 1-23, 2020. [10.4018/JDM.2020100101](https://doi.org/10.4018/JDM.2020100101).
- [12] H. B. Gebreweld, *Architecture Knowledge Representation and Communication Industry Survey*, 2023. [10.48550/arXiv.2309.11572](https://arxiv.org/abs/2309.11572). arXiv: [2309.11572](https://arxiv.org/abs/2309.11572). dirección: <https://arxiv.org/abs/2309.11572>.
- [13] J. E. T. Akinsola, A. O. Akinkunmi, I. M. Olaniyi, J. E. Efiog y E. A. Olajubu, “Multi-Criteria Decision-Making (MCDM) Approach for Software Architecture Selection in Cloud Computing Using Evidential Reasoning and Bayesian Inference Techniques,” *International Journal of Information Engineering and Electronic Business*, vol. 18, n.º 1, págs. 1-12, 2026. [10.5815/ijieeb.2026.01.01](https://doi.org/10.5815/ijieeb.2026.01.01).
- [14] G. Buchgeher, R. Ferreira, V. Geist, R. Weinreich y C. Nagl, “Using Architecture Decision Records in Open Source Projects – An MSR Study on GitHub,” *IEEE Access*, 2023. [10.1109/ACCESS.2023.3287654](https://doi.org/10.1109/ACCESS.2023.3287654).

Anexos

Anexo A: Estructura del repositorio (rama feature/data-layer)

```
GA-PracticaExperimental-Unidad-III/
|-- docs/
|   |-- adr/                        # ADR-001, ADR-002, ADR-003
|   |-- arquitectura/              # Diagramas C4 (PNG)
|   |-- evidencias/                # Capturas de pruebas, benchmark, etc.
|   |-- referencias/               # Material bibliografico de apoyo
|   '-- Anexo-A-estructura-repositorio.md
|-- src/
|   |-- main/
|   |   '-- java/.../
|   |       |-- entity/            # Usuario, Categoria, Entidad (JPA)
|   |       |-- repository/        # JpaRepository + Specifications
|   |       |-- service/           # EntidadService, AuthService
|   |       |-- controller/        # EntidadController, AuthController
|   |       |-- security/          # JwtProvider, JwtAuthFilter,
|   |       |                     # TokenBlacklistService
|   |       |-- config/            # SecurityConfig, RedisCacheConfig
|   |       |-- dto/               # Requests/Responses, PageResponse
|   |       '-- exception/         # Manejo global de errores
|   '-- test/                     # Pruebas unitarias del Repository
|-- target/
|   |-- surefire-reports/          # Resultados de ejecucion de tests
|   '-- jacoco.exec                # Reporte de cobertura (JaCoCo)
|-- docker-compose.yml             # PostgreSQL + Redis + backend
|-- pom.xml                        # Dependencias Maven
'-- README.md
```

Anexo C: Preguntas de Análisis

[Análisis] Speedup del benchmark de caché.

Sí. El umbral de referencia ($S > 2$) se supera: la consulta principal del PFC pasó de una media de 45,49 ms (consultando PostgreSQL con join a categorías y paginación) a 14,36 ms (respondida desde Redis), un speedup de 3,17x. Evaluando los tres criterios relevantes:

(1) costos de infraestructura: un contenedor Redis (redis:7-alpine) tiene una huella mínima de memoria y CPU frente al motor relacional, sin costo operativo relevante para un PFC universitario; (2) complejidad de despliegue: se añade un servicio más, resuelto con una entrada adicional en docker-compose.yml, sin necesidad de clúster ni sharding; (3) impacto de fallos de Redis en disponibilidad: el patrón cache-aside implementado es fail-open — si Redis no responde, la petición cae a consultar PostgreSQL directamente, por lo que una caída de Redis degrada el rendimiento pero no la disponibilidad del CRUD. Dado que $S = 3,17x$ supera el umbral y el riesgo de disponibilidad es bajo, se justifica mantener Redis en el stack. El speedup es moderado porque el dataset semilla es pequeño (55 registros) y corre en localhost, donde la latencia de red y disco es mínima; en un entorno con más datos y con la base de datos en un host distinto, la diferencia relativa tendería a ampliarse.

[Síntesis] Estrategia de invalidación de caché.

En el módulo principal del PFC, la estrategia de invalidación sigue el patrón cache-aside descrito en el ADR-003: la clave de caché se genera de forma dinámica combinando el filtro de búsqueda y el número de página utilizados en la consulta (por ejemplo, `entidad-listado::filtro-0`), de modo que cada combinación distinta de filtros y paginación se almacena como una entrada independiente en Redis.

Cuando se crea, actualiza o elimina una `Entidad`, el método correspondiente en `EntidadService` está anotado con `@CacheEvict(value = ".entidad-listado", allEntries = true)`, lo cual invalida *todas* las entradas del caché de listado, sin importar qué filtro o página tenían almacenada. Esta decisión es intencionalmente conservadora: como no se puede predecir de antemano qué combinaciones de filtro/página contienen el registro modificado, invalidar todo el grupo de claves evita servir datos desactualizados (stale cache), a costa de perder temporalmente el beneficio de la caché para el resto de combinaciones hasta que se vuelvan a consultar.

Respecto al riesgo de cache stampede, sí existe en este módulo: si la entrada de caché del filtro y página más consultados vence al mismo tiempo en que llegan varias peticiones simultáneas, todas esas peticiones detectarán un cache miss a la vez y consultarán PostgreSQL de forma simultánea, generando una carga innecesaria justo en el momento del vencimiento. Para un PFC de este tamaño (50 registros, tráfico bajo) el riesgo real es mínimo, pero la mitigación recomendada sería introducir un TTL con jitter aleatorio (por ejemplo, en vez de un TTL fijo de 300 segundos, usar un valor aleatorio entre 270 y 330 segundos por entrada), de forma que las entradas no venzan todas exactamente en el mismo instante, o alternativamente usar un mecanismo de bloqueo (mutex) que permita que solo la primera petición que detecta el

miss recalcula el dato, mientras las demás esperan ese resultado en lugar de consultar la base de datos por separado.

[Evaluación] Comparación del ADR-001 con la literatura.

El ADR-001 documenta la decisión de adoptar una arquitectura monolítica por capas (**Controller** → **Service** → **Repository** → **Entity**) para el PFC, justificada principalmente por el tamaño reducido del equipo (3 integrantes) y el riesgo de sobrecarga operacional que introducirían los microservicios sin una necesidad real de escalar de esa forma. Esta misma trayectoria está documentada en sistemas de producción reales: casos ampliamente citados en la literatura, como el de grandes plataformas que iniciaron su desarrollo como una aplicación monolítica y solo migraron hacia microservicios una vez que el tráfico y el tamaño del equipo de desarrollo crecieron lo suficiente como para justificar esa complejidad adicional. Estudios recientes sobre microservicios señalan que esta migración progresiva —comenzar monolítico y descomponer por partes— es el patrón más común en la industria, en lugar de diseñar microservicios desde el primer día [7]. De igual forma, comparaciones empíricas entre arquitecturas monolíticas y de microservicios confirman que el monolito ofrece mejor rendimiento y menor complejidad operacional cuando el volumen de tráfico aún no exige escalabilidad independiente por módulo, que es exactamente el escenario del PFC [8].

La diferencia principal entre el caso del ADR-001 y esos sistemas de producción es la escala: mientras que empresas como las citadas migraron a microservicios al enfrentar millones de usuarios concurrentes y decenas de equipos de desarrollo trabajando en paralelo, el PFC opera con datos de prueba (50 registros) y un solo equipo pequeño, por lo que la decisión monolítica no solo es válida sino la más adecuada en este momento.

En cuanto a qué aspectos del PFC podrían requerir una arquitectura diferente si el sistema escalara de verdad, el candidato más claro es el acoplamiento actual entre la lógica de negocio y el acceso a datos: si en el futuro se agregara, por ejemplo, un módulo de notificaciones en tiempo real (recordatorios automáticos, alertas), ese módulo sería el primer candidato natural para separarse como un servicio independiente, ya que su carga de trabajo (enviar notificaciones) es de naturaleza distinta a la del CRUD principal y no debería bloquear ni compartir el mismo ciclo de despliegue que el resto del sistema. De igual forma, si la base de datos PostgreSQL única se convirtiera en un cuello de botella por alto volumen de lectura, la arquitectura debería evolucionar incorporando una réplica de lectura, tal como se contempla en el análisis de escalabilidad horizontal del documento del PFC, antes de considerar siquiera una descomposición completa en microservicios.

[Evaluación] Cobertura de pruebas del Repository.

La cobertura alcanzó 100 por ciento (instrucciones y ramas) en la capa Repository, medida sobre la clase EntidadSpecifications mediante JaCoCo, con 21 pruebas en total (17 en EntidadRepositoryTest, 4 en CategoriaRepositoryTest). Aun así, hay tipos de bugs que estas pruebas con base de datos en memoria (H2) no capturan: diferencias de comportamiento entre H2 y PostgreSQL en producción (funciones SQL específicas, tipos de datos, collation en comparaciones LIKE), problemas de concurrencia bajo carga real, y el rendimiento real de las queries con volúmenes grandes de datos. Casos de prueba adicionales de bajo costo de mantenimiento: pruebas de valores límite en stockMinimo y precioMinimo/precioMaximo; prueba de filtro combinado con resultado vacío (para confirmar que el AND de las Specification no degrada a un OR); prueba de integridad referencial al guardar una Entidad con una Categoria no persistida; prueba de unicidad de email en UsuarioRepository. Bugs que la prueba unitaria del Repository no captura: errores de configuración de Flyway/producción (las pruebas corren contra H2 con ddl-auto=create-drop, no contra el esquema real generado por las migraciones sobre PostgreSQL); problemas de concurrencia (race conditions, bloqueos optimistas/pesimistas) al no ejercitarse en un test secuencial; el comportamiento real de @Cacheable/@CacheEvict contra un Redis real, que se valida en el benchmark del apartado 5.4, no en JUnit; y problemas de rendimiento a escala real, dado que el dataset de prueba es pequeño (4-5 registros por test) y no revela problemas de N+1 o planes de ejecución lentos que solo aparecen con miles de registros. Pruebas de integración contra PostgreSQL real (Testcontainers) y pruebas de carga complementarían esta cobertura unitaria.